GRAM■ JYOTHI RURAL ELECTRICITY CO-OPERATIVE
C PROGRAMMING ASSIGNMENT
Tasks 1–4: Implementation, Testing, Analysis and Design Challenge

Name: BAVADARANI S

Register Number:1912260562

GitHub Repository Link: _____https://github.com/bavadarini28/ CSA0209-1912260562-BAVADARANI-S-ASSIGNMENT-1.git

How to Use This Document

This is an editable working report and coding guide based on the assignment brief you provided. Replace the blanks with your details, execute the programs on your own computer, and enter your actual results and screenshots. Do not report sample or predicted measurements as if you measured them.

Create a GitHub repository named CSA02-<RegisterNumber>-Assignment1.

Create folders Task01, Task02, Task03 and Task04.

Save each program in the matching folder. Keep input files and screenshots of actual runs there.

Use GCC with warnings enabled and record the exact compiler version and command.

Commit progressively, with at least one commit for each task.

Put the working GitHub link on this cover page and in the submission field.

Scenario and Assumptions

The co-operative bills households using meter readings and a slab tariff. This report uses the example tariff from the brief: first 100 units free, units 101–200 at Rs 2.25 per unit, and units above 200 at Rs 4.50 per unit. Because the brief does not specify the fixed charge, the starter implementation assumes Rs 30.00. Confirm this assumption with your instructor or change the single constant if a different charge is required.

Money is represented as integer paise: Rs 2.25 = 225 paise and Rs 4.50 = 450 paise. This avoids ordinary binary floating-point rounding in the bill calculation.

TASK 1 – Implement and Test the Billing Program (K3)

File to save as: Task01/billing.c

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

#define FREE_UNITS 100u
#define SECOND_SLAB_LIMIT 200u
#define SECOND_SLAB_PAISE 225LL
#define THIRD_SLAB_PAISE 450LL
#define FIXED_CHARGE_PAISE 3000LL
#define METER_MAX 65535u

static uint32_t calculate_units(uint16_t previous, uint16_t current)
{
    /* Unsigned 16-bit subtraction gives the rollover consumption modulo 65536. */
    return (uint16_t)(current - previous);
}

static int64_t calculate_bill_paise(uint32_t units)
{
    int64_t energy = 0;

    if (units > FREE_UNITS) {
        uint32_t second = (units < SECOND_SLAB_LIMIT)
                        ? units - FREE_UNITS
```

```
                        : SECOND_SLAB_LIMIT - FREE_UNITS;
        energy += (int64_t)second * SECOND_SLAB_PAISE;
    }

    if (units > SECOND_SLAB_LIMIT) {
        energy += (int64_t)(units - SECOND_SLAB_LIMIT) * THIRD_SLAB_PAISE;
    }

    return FIXED_CHARGE_PAISE + energy;
}

static void print_money(int64_t paise)
{
    printf("Rs %" PRId64 ".%02" PRId64 "\n", paise / 100, paise % 100);
}

int main(void)
{
    long long previous_input, current_input;
    uint16_t previous, current;
    uint32_t units;
    int64_t bill;

    printf("Enter previous reading (0-65535): ");
    if (scanf("%lld", &previous_input) != 1 ||
        previous_input < 0 || previous_input > METER_MAX) {
        fprintf(stderr, "Invalid previous reading.\n");
        return 1;
    }

    printf("Enter current reading (0-65535): ");
    if (scanf("%lld", &current_input) != 1 ||
        current_input < 0 || current_input > METER_MAX) {
        fprintf(stderr, "Invalid current reading.\n");
        return 1;
    }

    previous = (uint16_t)previous_input;
    current = (uint16_t)current_input;

    /* A decrease is treated as rollover only when it crosses the meter limit. */
    if (current < previous && previous < 65000u) {
        fprintf(stderr, "Invalid reading order: decrease is not an accepted rollover.\n");
        return 1;
    }

    units = calculate_units(previous, current);
    bill = calculate_bill_paise(units);

    printf("Units consumed: %" PRIu32 "\n", units);
    printf("Bill: ");
    print_money(bill);
    return 0;
}
```

Compile and Run

gcc -std=c11 -Wall -Wextra -pedantic Task01/billing.c -o Task01/billing

./Task01/billing

Note: The sample code accepts a lower current reading as rollover only when the previous reading is at least 65000. If your instructor expects a different rollover rule, document it and adjust the validation condition.

Test Plan and Results

Run each case. Fill the Actual Result and Pass/Fail columns from your own program output.

Important: The example code's rollover validation uses a practical threshold for demonstration. A 16-bit counter rollover is mathematically defined modulo 65536, but the program cannot always distinguish a genuine rollover from bad data using only two readings. State the rule you adopt and test it.

Five-Bill Hand Calculation

Use these sample consumption values to check the arithmetic; record your program's actual result.

Expected total = Rs 937.50. Verify this by adding the five bills and by running your own test.

Alternative: Arrays and a Loop

```
/* Conceptual alternative: slab boundaries and rates are stored in arrays.
   A production version must define clearly whether each rate applies to
   the whole slab or only units within that slab. */
const uint32_t upper[] = {100u, 200u, UINT32_MAX};
const int64_t rate_paise[] = {0LL, 225LL, 450LL};
/* Loop through slabs, calculate the units in each slab, and add
   slab_units * rate_paise[i]. */
```

With a fixed number of if statements, the calculation is O(1) time and O(1) extra space. An array-based loop over s slabs takes O(s) time and O(s) storage for the tariff arrays. The array design is easier to revise or extend to more slabs without adding many new conditional branches.

TASK 2 – Compare Expectations with Execution (K4)

File to save as: Task02/types_measure.c

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdint.h>
#include <inttypes.h>

int main(void)
{
    printf("sizeof(int) = %zu, range = %d to %d\n",
           sizeof(int), INT_MIN, INT_MAX);
    printf("sizeof(long) = %zu, range = %ld to %ld\n",
           sizeof(long), LONG_MIN, LONG_MAX);
    printf("sizeof(long long) = %zu, range = %lld to %lld\n",
           sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("sizeof(float) = %zu, FLT_DIG = %d\n", sizeof(float), FLT_DIG);
    printf("sizeof(double) = %zu, DBL_DIG = %d\n", sizeof(double), DBL_DIG);
    printf("sizeof(int64_t) = %zu\n", sizeof(int64_t));

    /* Demonstration: add the sample bill in integer paise and float rupees. */
    const int bill_count[] = {4000, 60000, 800000};
    const int64_t sample_bill_paise = 48000; /* Rs 480.00 per household */

    for (size_t i = 0; i < sizeof bill_count / sizeof bill_count[0]; ++i) {
        int64_t total_paise = (int64_t)bill_count[i] * sample_bill_paise;
        float total_rupees = (float)bill_count[i] * 480.00f;
        printf("%d bills: paise total = %" PRId64
               ", float total = %.2f rupees\n",
               bill_count[i], total_paise, total_rupees);
    }
    return 0;
}
```

Compile and Run

gcc -std=c11 -Wall -Wextra -pedantic Task02/types_measure.c -o Task02/types_measure

./Task02/types_measure

This program uses Rs 480.00 as a controlled example bill for every household, so the scale comparison is repeatable. It does not claim every household will actually have that bill. Record the exact output from your system.

Analysis to Complete After Running

Paise total: integer multiplication gives an exact total as long as the chosen integer type does not overflow.

Float total: float has limited significant decimal precision. Compare the printed value with total_paise / 100.0.

int, long and long long: use the printed ranges to decide whether a worst-case district total in paise fits. Do not actually overflow a signed type; signed overflow is undefined behaviour in C.

sizeof: record the byte sizes reported by your compiler. C guarantees minimum ranges, not identical byte sizes on every platform.

double alternative: double usually has much wider range and better precision than float, but it is still binary floating point and is not a substitute for exact integer paise when exact currency is required.

For this fixed-bill example, the exact totals are calculated mathematically in the table; the float observation must come from your run.

TASK 3 – Recommend and Defend the Final Design (K5)

Recommended Design

Meter readings: uint16_t when the meter specification is exactly a 16-bit unsigned counter; validate input before conversion.

Consumption: uint32_t, because rollover consumption may exceed the current reading and should be represented explicitly.

Money: int64_t in paise for each bill and the aggregate total. Confirm that the maximum plausible district total fits the chosen type.

Tariff constants: centralize them in one header or one tariff structure rather than repeating numeric values in multiple functions.

Preprocessor: use include guards and #define for compile-time constants. Use a debug macro only for optional diagnostic output.

Validation: check scanf results, numeric ranges, negative values, and unexpected reading decreases.

Tariff revision with compiled constants: update the centralized tariff definitions, run all boundary tests, compile, and redistribute the tested executable. A configuration file can avoid recompilation, but it needs parsing, validation, error handling, and a clear policy for missing or invalid values.

Configuration File Alternative and Complexity

Reading t tariff entries at startup takes O(t) time and O(t) memory if all entries are stored. For a small tariff table, this overhead is usually small compared with processing hundreds of thousands of households. The benefit is that authorized tariff changes can be deployed without recompiling; the risk is malformed or inconsistent configuration, so validate slab order, non-negative rates, and units before accepting the file.

Evidence to add: paste your actual Task 1 test results and Task 2 measurements here before final submission.

TASK 4 – Smart-Meter Design Challenge (K5)

Decision: MODIFY the Original Design

Retain integer-based monetary arithmetic, validation, centralized tariff data, and clear separation between input, tariff calculation, and reporting. Modify the input model to handle timestamped watt-hour readings every 15 minutes and use a tariff table loaded from a configuration file. A full redesign is not required for the basic calculation, but data ingestion and storage need to be redesigned for scale.

Smart-Meter Requirements

There are 35,040 fifteen-minute intervals per house per year (365 × 24 × 4).

Use a sufficiently wide unsigned integer type for cumulative watt-hours and validate counter rollover based on the actual meter specification.

Compute interval energy from consecutive readings; convert watt-hours to billing units using a defined, consistent rule.

Apply a 20% surcharge to energy used from 6 PM through 10 PM. Define timezone, timestamp boundaries, missing intervals, and daylight-saving policy if applicable.

Load time-of-day rates from a configuration file at startup, validate it, and log which tariff version was used for each billing period.

For m interval records, a chronological streaming calculation can take O(m) time and O(1) working memory beyond tariff data. Storing all intervals takes O(m) memory.

Illustrative C Design for One Interval

```
  #include <stdint.h>
  #include <stdbool.h>

  /* Illustrative helper: energy is in Wh; the rate is paise per kWh.
     Real code must validate timestamps and use a sufficiently wide total. */
  static int64_t charge_paise(uint64_t energy_wh,
                              int64_t rate_paise_per_kwh,
                              bool peak_period)
{
      int64_t rate = rate_paise_per_kwh;

      /* 20% peak surcharge represented as a rational multiplier.
         This example assumes the resulting paise is rounded down.
         The production policy must define the required rounding rule. */
      if (peak_period) {
          rate = (rate * 120) / 100;
      }

      /* Exact fractional paise may occur; define rounding consistently.
         This simple integer expression truncates, so replace it with the
         co-operative's documented rounding policy if required. */
      return (int64_t)(energy_wh * (uint64_t)rate / 1000u);
  }
```

This helper is an illustration, not a complete production smart-meter system. Before submission, explain the rounding rule for fractional paise, validate the multiplication range, and show how timestamps determine whether an interval falls within 18:00–22:00. For intervals crossing 6 PM or 10 PM, split the interval energy proportionally only if the measurement model supports that calculation; otherwise document the meter's interval alignment assumptions.

Final Testing Checklist

Task 1: execute every test, fill actual results, and save terminal screenshots.

Task 2: record compiler version, sizeof output, ranges, and totals.

Task 3: ensure recommendation refers to actual results rather than unsupported claims.

Task 4: document time-of-day boundaries, tariff-file validation, rounding, and scale.

README.md: name, register number, GCC version, exact commands, run instructions, and short result for each task.

GitHub: one folder per task, at least one meaningful commit per task, and a working repository link on the cover page.

Case	Previous	Current	Expected result	Actual / Pass?
Zero units	1000	1000	0 units; Rs 30.00	
Exactly 100 units	1000	1100	100 units; Rs 30.00	
Exactly 200 units	1000	1200	200 units; Rs 255.00	
Above 200 units	1000	1250	250 units; Rs 480.00	
Rollover	65530	5	11 units; Rs 30.00	
Negative reading	-1	100	Invalid input; non-zero exit	
Letters	abc	100	Invalid input; non-zero exit	
Decrease, not rollover	1200	1100	Invalid reading order	

House	Units consumed	Expected bill
A	0	Rs 30.00

House	Units consumed	Expected bill
B	100	Rs 30.00
C	150	Rs 142.50
D	200	Rs 255.00
E	250	Rs 480.00

Bills	Exact paise total	Exact rupee total	Observed float output
4,000	192,000,000	Rs 1920000.00	Fill from run
60,000	2,880,000,000	Rs 28800000.00	Fill from run
800,000	38,400,000,000	Rs 384000000.00	Fill from run